

Оракул — личный AI-астролог в Telegram

Документация проекта: идея, продукт, маркетинг, разработка, дизайн, монетизация, операции, право

Содержание:

1. Обзор (README)
2. Идея продукта
3. Продукт и функциональность
4. Маркетинг
5. Разработка
6. Дизайн и бренд
7. Frontend-ТЗ
8. Бизнес и монетизация
9. Операции и эксплуатация
10. Право и приватность



Обзор (README)

Оракул — личный AI-астролог в Telegram

Личный астролог, таролог, нумеролог, наставница практик и дневник — в одном чате. Расчёты делает код, модель их трактует: натальная карта по Swiss Ephemeris, честный случайный выбор карт Таро, Матрица Судьбы по формулам. Модель **не выдумывает** планеты и карты — отсюда «правдивость», на которой держится доверие.

Сервис полного цикла: Telegram-бот (aiogram) + Mini App (FastAPI) + админ-панель + каналы продвижения (гороскопы по знакам, PDF-разборы на маркетплейсах, рефералка). Для русскоязычных женщин 25–45, которые хотят не «гороскоп для всех», а личный проводник, который помнит именно её.

Быстрый старт

```
cd oracle
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env          # BOT_TOKEN, ADMIN_ID, WEBAPP_URL
python -m app.bot.main        # бот
uvicorn app.api.main:app --port 8080 # API / Mini App
```

Бот работает сразу: без ключей — офлайн-режим на реальных расчётах; с ключом Claude/OpenAI/локального сервера — полный агентный режим с цепочкой провайдеров. Проверка проекта одной командой: `python -m scripts.selfcheck`.

Ключевое

- **Шесть агентов:** Оракул (Лилит), Таролог, Астролог, Нумеролог, Наставница практик, Хранитель дневника — с памятью и 12 инструментами расчёта.
- **Астрология:** натальная карта, транзиты, фаза Луны, лунный календарь, синастрия, деловые окна.
- **Таро:** 78 карт, 9 раскладов, честный рандом, 3D-переворот, история с отметкой «сбылось».
- **Матрица Судьбы,** практики и мантры (12 программ со стриком), дневник, карьера.
- **Гороскопы по 12 знакам** с автопостингом в каналы-спутники — бесплатный канал трафика.
- **Монетизация:** триал 30 дней, подписки за Stars и web (Paddle), Кристаллы ✦, одиночные расклады, PDF-разборы.
- **Рост:** двухуровневая рефералка, карточки для сторис, SEO-лендинг.

- **Безопасность:** кризисный протокол, антифлуд, идемпотентные платежи, `/delete_me`; Mini App под строгим CSP без `unsafe-inline`: 0 inline-хендлеров (все действия на делегировании `data-act`), серверный текст экранируется белым списком `<b>/<i>`, шрифты селфхостятся в `miniapp/fonts/`, ассеты с cache-busting `?v=1`.

Карта документации (по отделам)

Документация — это описание продукта, а не этапов разработки. Каждый отдел начинает со своего файла:

Документ	Для кого	О чём
docs/IDEA.md	все	Идея, проблема, уникальность, ценности, видение
docs/PRODUCT.md	продакт, поддержка	Функциональность, агенты, скиллы, Mini App, фиче-флаги
docs/MARKETING.md	маркетинг	Позиционирование, аудитория, каналы, контент, запуск, KPI
docs/DEVELOPMENT.md	разработка	Стек, архитектура, структура, LLM-слой, безопасность, тесты
docs/DESIGN_SPEC.md	дизайн	«Дорогая мистика», токены, Mini App, доступность
docs/MONETIZATION.md	бизнес	Тарифы, товары, платежи, юнит-экономика, метрики
docs/OPERATIONS.md	DevOps	Деплой, мониторинг, бэкапы, запуск, SLO, масштабирование
docs/LEGAL.md	юрист	Приватность, согласие, оферта, возвраты, кризисный протокол

Структура репозитория

- `app/bot/` — aiogram: онбординг, чаты, Таро, практики, профиль, оплата
- `app/core/` — агенты, расчёты (астрология/Таро/Матрица), память, безопасность, скиллы
- `app/api/` + `web/` — FastAPI, Mini App, лендинг, robots/sitemap
- `miniapp/` — Mini App (chat-first): `app.js`, `styles.css`, `fonts/` (самохостинг шрифтов)
- `app/services/` — правила продукта: лимиты, деньги, чат, практики, гороскопы, планировщик
- `app/repo/`, `app/data/` — SQL-слой, схема, миграции, сид каталога
- `app/pdfgen/` — PDF-разборы (товар для маркетплейсов)
- `admin/` — админ-панель: каталог, контент, флаги, рассылки, аналитика, аудит
- `infra/` — Docker, Caddy, бэкапы · `scripts/` — selfcheck, промокоды, PDF · `tests/` — pytest
- БД: SQLite в `data/oracle.db` (создаётся сама, режим WAL)

Полезные команды

```
python -m scripts.selfcheck          # проверка проекта
pytest tests/ -q                     # автотесты
python -m scripts.gen_promo 20 --days 30 --batch etsy-1  # промокоды
python -m scripts.gen_pdf --name Анна --date 21.06.1999 --time 14:30 --city Казань #
PDF-разбор
```

Деплой — `docker compose -f infra/docker-compose.yml up -d --build` (подробно в `docs/OPERATIONS.md`).

Идея продукта

Идея продукта

О чём этот проект

«Оракул» — личный AI-астролог в Telegram. Это не очередной «бот-гадалка», а сервис полного цикла: бот, где живёт общение, Mini App с витриной и картой, админ-панель для управления продуктом и каналы продвижения, которые сами приводят трафик.

Ключевой принцип, на котором держится всё: **расчёты делает код, модель только тракует**. Натальная карта считается по Swiss Ephemeris (те же эфемериды, что у профессиональных астрологов), карты Таро выбираются криптографическим генератором случайных чисел, арканы Матрицы Судьбы считаются по формулам. Языковая модель не выдумывает планеты и карты — она получает готовый расчёт и объясняет его живым языком. Отсюда «правдивость», на которой строится доверие.

Проблема

Рынок эзотерики в Telegram огромен, но почти весь он состоит из двух крайностей:

- **«Болталки» на промптах:** один большой LLM-промпт, который сочиняет планеты, дома и «расклады» на ходу. Пользовательница с астрологическим образованием ловит несостыковку за первое же сообщение — и доверие к сервису в целом умирает.
- **Общие тексты:** гороскоп «для всех Овнов» из открытых источников, без даты рождения, без памяти, без диалога.

Живой астролог решает проблему качества, но он дорог, работает по записи и не помнит клиентку между сессиями.

Пользовательнице не нужен «бот-говорилка». Ей нужен личный проводник, который знает её дату рождения, помнит, что она рассказывала две недели назад, и отвечает конкретно про её жизнь, а не про «Овнов вообще».

Решение

Оракул — это шесть специализированных агентов в одном чате: личный Оракул, Таролог, Астролог, Нумеролог, Наставница практик и Хранитель дневника. Вместе они закрывают основные запросы аудитории:

- понять себя (натальная карта, Матрица Судьбы, предназначение);
- отношения (синастрия, расклады на любовь);
- деньги и карьеру (деловые окна, расклады на работу, разбор предназначения);
- ежедневную практику (прогноз дня, практики и мантры со стриком);
- рефлекссию (дневник с вечерним вопросом и месячным отчётом).

Дополнительно сервис умеет быть продуктом, а не только чатом: PDF-разборы для маркетплейсов, картинки-карточки для сторис, каналы-спутники с ежедневными гороскопами.

Чем отличается от конкурентов

1. **Настоящие расчёты.** Модель не выдумывает — это главный козырь, на нём построено и позиционирование, и доверие.
2. **Шесть специализированных агентов** вместо одного «мастера на все руки». У каждого узкий набор инструментов и свои правила — это качество ответов, а не декорация.
3. **Долгая память по смыслу.** Бот помнит факты из переписки недельной давности, дедуплицирует их по смыслу и ищет под текущий вопрос.
4. **Полноценный продукт**, а не болталка: монетизация, витрина, реферальная программа, каналы трафика, админ-панель, аналитика.

Целевая аудитория

Русскоязычные женщины 25–45 лет. Интересуются гороскопами, Таро, практиками и саморазвитием; платят за персонализацию и качество; устали от «одних и тех же общих гороскопов». Ищут не развлечение, а понимание себя и опору в принятии решений.

Ценности продукта

- **Правдивость.** Код считает, модель объясняет. Никаких выдуманных планет и карт.
- **Забота.** Тёплый, человеческий язык. Кризисный протокол: сообщение о суициде, самоповреждении или насилии не доходит до модели — человек получает поддержку и контакты помощи, лимит не тратится.
- **Уважение к данным.** Дату рождения, переписку и память можно стереть одной командой `/delete_me`. Политика приватности — в `docs/LEGAL.md`.
- **Без манипуляций.** Не пугаем «тяжёлыми» планетами, не обещаем чудес, не давим на страх. Прогнозы — для самопознания и вдохновения, дисклеймер прямо в настройках сервиса.

Видение

Каждая русскоязычная женщина, которая хочет понять себя и свой путь, имеет личного Оракула: он помнит её, считает честно и говорит по-человечески — 24/7, за цену ниже одной консультации живого астролога.

Что продукт осознанно не делает

- Не предсказывает беды, болезни и смерть.
- Не заменяет врача, психолога, юриста или финансового консультанта.
- Не использует страх и «сглазы» как инструмент продажи.

- Не распознаёт голос в онбординге без явной надобности — данные рождения вводятся текстом, это осознанное решение по UX и приватности.

Статус и куда смотреть дальше

Проект — в стадии доведения до боевого запуска. Техническая часть (архитектура, стек, тесты) — `docs/DEVELOPMENT.md`, дизайн — `docs/DESIGN_SPEC.md`, монетизация и экономика — `docs/MONETIZATION.md`, маркетинг и запуск — `docs/MARKETING.md`, эксплуатация — `docs/OPERATIONS.md`, юридический пакет — `docs/LEGAL.md`.

Продукт и функциональность

Продукт и функциональность

Обзор

Оракул состоит из трёх поверхностей:

- **Telegram-бот** (aiogram) — онбординг, чаты с агентами, Таро, практики, профиль, оплата за Stars, рассылки.
- **Mini App** (FastAPI + встроенный фронтенд) — chat-first: четыре экрана (Сегодня · Агенты · Чат · Профиль). Чат — главный инструмент, функции агентов живут кнопками прямо в диалоге. Открывается из бота и из листингов Telegram.
- **Админ-панель** — управление без деплоя: каталог тарифов и товаров, контенты (тексты, персоны, расклады, практики), фиче-флаги, рассылки, аналитика, аудит действий, контакты кризисной поддержки.

Шесть агентов

У каждого агента свой характер, свой голос и **узкий набор инструментов** — специализация это качество, а не украшение.

Агент	Имя	Эмодзи	Роль	Инструменты
Личный Оракул	Лилит	▪	Главный агент: знает карту, помнит всё, отвечает на любые вопросы	карта, транзиты, луна, Таро, Матрица, синастрия, карьерные окна, практики, память, дневник
Таролог	Мадам Ленорман		Только карты, школа Райдера-Уэйта	Таро, память
Астролог	Урания	▪	Натальная карта, транзиты, синастрия, выбор дат	карта, транзиты, луна, карьерные окна, синастрия, память
Наставница практик	Ная	ॐ	Практики и мантры, трекер дней подряд	практики, луна, дневник, память
Нумеролог	Пифия		Матрица Судьбы: предназначение, деньги, род	Матрица, память
Хранитель дневника	Мнемо		Помнит записи, замечает динамику, задаёт один вопрос	дневник, память

Образ речи главного агента выбирает сама пользовательница: «Мудрая подруга», «Таинственная ведунья» или «Духовный наставник» — это меняет только манеру, не суть.

Принцип «код считает, модель трактует»

Модель вызывает 12 детерминированных инструментов (скиллов), которые исполняет код:

- `draw_tarot` — честный выбор карт (криптографический random);
- `get_chart` — натальная карта по Swiss Ephemeris;
- `get_transits` — транзиты на сегодня/период;
- `get_moon_week` — фазы Луны и лунный календарь на неделю;
- `get_career_windows` — деловые окна: когда действовать, когда молчать;
- `get_matrix` — Матрица Судьбы (арканы, линии, предназначение);
- `get_compatibility` — синастрия пары (скоринг + разбор);
- `list_partners` — сохранённые партнёры;
- `suggest_practice` — практика из каталога под запрос;
- `save_memory` / `recall_memory` — долгая память по смыслу;
- `recall_diary` — дневниковые записи.

Правила трактовки (например, школа Райдера-Уэйта для Таро) подмешиваются к результату расчёта, чтобы модель разбирала данные по правилам школы, а не «как получится». Модель **никогда не выдумывает** карты, планеты и арканы — она получает готовый расчёт и объясняет его.

Направления

Астрология. Настоящая натальная карта (Swiss Ephemeris через kerykeion): планеты по знакам и домам, асцендент, аспекты. Транзиты, фаза Луны, лунный календарь, синастрия пары, деловые окна для карьерных решений.

Таро. 78 карт, честный `secrets` -рандом, 9 раскладов: «Одна карта», «Прошлое-Настоящее-Будущее», «На отношения», «Выбор из двух», «Деньги и дело», «Карьера и путь», «Проблемы на работе», «Кельтский крест», «Колесо года». Расклад начинается с вопроса: клиентка формулирует его картам, вопрос сохраняется в раскладе и передаётся в LLM-трактовку. 3D-переворот карт, история раскладов с отметкой «сбылось».

Матрица Судьбы. Арканы по дате рождения: предназначение, денежная и родовая линии, октаграмма, полный разбор.

Практики и мантры. Четыре категории (мантры, деньги, любовь, энергия), 12 программ с шагами, расписанием по дням, стриком, напоминаниями и «знаками продвижения».

Карьера. Расклады на карьере и рабочие конфликты, разбор предназначения, деловые окна «когда действовать, когда молчать».

Дневник. Стрик, вечерний вопрос от Оракула, месячный отчёт «что показала Вселенная». В Mini App реализован как «Книга судьбы».

Гороскопы по 12 знакам. Один общий текст на знак в сутки (не по клиентке) собирается планировщиком и публикуется в каналы-спутники — бесплатный канал трафика и SEO-подпитка.

PDF-разборы. Товар для маркетплейсов: покупательница получает PDF с разбором карты, внутри — «золотой билет», промокод на 30 дней VIP.

Память

Бот хранит факты из переписки и дедуплицирует их **по смыслу** (не по тексту), ищет под текущий вопрос и собирает сводку. У каждого тарифа своя глубина памяти (5–80 фактов). Поиск — по словам, а при включённом флаге `semantic_memory` — по эмбедингам.

Mini App: chat-first, четыре экрана

- **Сегодня** — статичная база: прогноз, карта дня, фаза Луны, кольцо агентов. Всё, что требует вопроса, происходит в чате.
- **Агенты** — карточки шести агентов с кнопками-функциями (feature-чипсы), которые открывают чат и запускают функцию прямо в диалоге.
- **Чат** — главный инструмент: вопросы агенту и фичи-виджеты в ленте (расклад Таро с вопросом, натальная карта, лунная неделя, Матрица, совместимость).
- **Профиль** — хранилище результатов: натальная карта (строится в чате и сохраняется), последние расклады, купленные разборы, память.

API повторяет правила бота: без активной подписки — 402, сверх лимита — 429. Прогноз дня кешируется на сутки — бот и Mini App показывают один текст и не оплачивают генерацию дважды.

Фиче-флаги

Каждая механика включается флагом в админке без деплоя: витрина в Mini App, мультиагентность, голосовые вопросы, утренние прогнозы, отчёты недели/месяца, практики, двухуровневая рефералка, отметка «сбылось», web-оплата, бесплатный план после триала, семантическая память, гороскопы, автопостинг в каналы, озвучка прогнозов, напоминания о практиках, карточки для сторис. Это позволяет раскатывать фичи по аудитории постепенно и мгновенно откатывать.

Продуктовые правила

- Триал 30 дней полного доступа (по промокоду) → бесплатный план «Искра» с картой дня и одним вопросом в неделю.
- Лимиты — игровая механика: «пламя» в интерфейсе и кнопка «Спросить Оракула» (оплата/Кристаллы), а не дверь в лицо.
- Экстренный вопрос сверх лимита стоит Кристаллы (20 ✦).
- «Сбылось» на раскладах — обратная связь качества и контент для кейсов.

Подробности о тарифах и товарах — в `docs/MONETIZATION.md` , о дизайне и фронтенде Mini App — в `docs/DESIGN_SPEC.md` и `docs/FRONTEND_TZ.md` .

Маркетинг

Маркетинг

Позиционирование

Личный AI-астролог 24/7, который **помнит тебя** и делает **настоящие расчёты**.

Один слоган продукта: «Личный AI-астролог, который знает именно тебя».

Главный маркетинговый козырь — **честность расчётов**: карта считается по Swiss Ephemeris, Таро — честный случайный выбор, Матрица — по формулам. Модель ничего не выдумывает. Это отличие от «болталок»-конкурентов, которое можно доказывать фактом, а не обещанием.

Целевая аудитория

Русскоязычные женщины 25–45, интересующиеся гороскопами, Таро, практиками и саморазвитием. Болевые точки: «общие гороскопы не про меня», «не знаю, как поступить», «хочу понять себя», «хочу опору в принятии решений». Покупают персонализацию, качество и чувство, что их слушают.

Оффер

- **Бесплатный вход без карты**: карта дня + один вопрос в неделю — низкий порог, ценность с первого дня.
- **Триал 30 дней VIP** промокодом (в т.ч. через «золотой билет» в PDF).
- **VIP с трёх вопросов в день и полной памятью** — «личный астролог 24/7 дешевле одной консультации».
- **Отдельные товары без подписки**: одиночные расклады, разборы карты/Матрицы/синастрии, карьерный разбор.

Каналы привлечения (бюджет ≈ 0)

Приоритет — каналам, которые уже встроены в продукт:

1. **Каналы-спутники с гороскопами**. 12 каналов по знакам, ежедневный автопостинг. Это бесплатный ежедневный трафик и SEO-подпитка: $12 \times 30 = 360$ уникальных постов в месяц. Механика: подписчик видит гороскоп и переходит в бота за «своим».
2. **PDF-разборы на маркетплейсах**. Товар с обложкой и описанием под ключи; внутри — «золотой билет» (промокод на 30 дней VIP). Тёплый трафик, CAC ≈ себестоимость разбора. Партия = листинг, по ней в панели видно, какой листинг приводит платящих.

3. **Рефералка 2 уровней.** Бонус за **оплату**, а не регистрацию — антинакрутка. Пригласившая получает Кристаллы, есть и бонус 2-го уровня («подруга подруги»), реферальный revenue-share.
4. **Инфлюенсеры и эзо-блогеры.** Бартер на промокоды-билеты — CAC почти ноль.
5. **Органикa/SEO.** Публичный лендинг (`web/landing.html`) с OG и JSON-LD, под ключевые запросы («AI астролог», «натальная карта бесплатно», «таро на отношения», «матрица судьбы»). Роуты `/robots.txt` и `/sitemap.xml` готовы.

Контент и tone of voice

Один голос на весь продукт — бот, Mini App, PDF, каналы, лендинг: тёплый, «мистический», но без манипуляций и запугивания.

- **Контент-план каналов-спутников:** ежедневный гороскоп + редкие разборы (витрина качества) + кейсы «что было написано в прогнозе — и случилось» (с согласия).
- **Соцдоказательство:** отметка «сбылось» на раскладах → отзывы-кейсы «мне Оракул подсказал...».
- **Трекинг каналов:** диплинк `t.me/<бот>?start=<ключ>` на каждом лендинге/листинге даёт `source` — видно, какой канал привёл платящих.
- **Ретеншн-рассылки** (админка): предупреждение за 2 дня до конца подписки, возврат после окончания, активация триала, апселл guide → vip.

Бренд

- Название: «Оракул», лицо продукта и главный агент — Лилит.
- Айдентика: «Дорогая мистика» — ночь, золото-шампань, серифный шрифт для заголовков, живая Луна как фирменный знак. Подробно — в `docs/DESIGN_SPEC.md`.
- Использование эмодзи в хrome запрещено — только SVG-иконки. В текстах-каналах эмодзи допустимы в меру.

KPI маркетинга

Целевые вехи первого месяца (исходные ориентиры ниши):

- Активация: $\geq 60\%$ дошедших от `/start` до первого прогноза.
- Платная активация: триал → оплата $\geq 5\%$; конверсия витрины $\geq 2\%$ на активных.
- Ретеншн: $D7 \geq 25\%$, $D30 \geq 12\%$.
- Стоимость: $CAC < LTV/3$ (формулы — в `docs/MONETIZATION.md`).

Запуск (день X)

- **Предзапуск (7-14 дней):** тизеры в каналах-спутниках, предзапись, раздача промокодов блогерам, подготовка листингов PDF.

- **День X:** публикация в каналах-спутниках, пост инфлюенсера, рассылка по базе. Ожидаемый наплыв — он и есть цель нагрузочной подготовки в `docs/OPERATIONS.md`.
- **День X+1..3:** следить за конверсией и себестоимостью, оперативно гасить жалобы, **не менять** цены и механики без данных.
- **После стабилизации:** A/B-тесты триала, цен, оффера и реферального оффера.

Чего маркетинг не должен делать

- Не переводить убеждающие тексты в опустошенную гляцевую риторику: продукт продаёт честность расчётов, а не «волшебные гарантии».
- Не давать обещаний о здоровье, богатстве и гарантиях — это и этика, и риск, и противоречие оффере в `docs/LEGAL.md`.

Разработка

Разработка

Техническая документация: стек, архитектура, структура репозитория, запуск, ключевые подсистемы и инженерные решения. Для понимания «как устроено и как разрабатывать». Эксплуатация (деплой, мониторинг, бэкапы) — в `docs/OPERATIONS.md`.

Стек

- **Python 3.13+**, ~15 600 строк кода в `app/`.
- **Telegram-бот**: `aiogram` (FSM для онбординга, клавиатуры, инлайн-оплата за Stars).
- **API / Mini App**: FastAPI + встроенный фронтенд (HTML/CSS/JS), сервится тем же процессом.
- **БД**: SQLite (WAL), один файл, схема и миграции в `app/data/`. 40 таблиц.
- **Астрология**: Swiss Ephemeris через `kerykeion` (5.x), геокодирование `geopy`. Тяжёлые библиотеки необязательны: без них бот работает в лайт-режиме астрологии.
- **LLM**: Claude (Anthropic), OpenAI и любой OpenAI-совместимый сервер (vLLM, LM Studio, Ollama) через единый слой.
- **Инфраструктура**: Docker Compose, Caddy (авто-https), Sentry, UptimeRobot.
- **Тесты**: pytest, 250 автотестов + самопроверка `scripts/selfcheck.py`.

Архитектура

```
Telegram ↔ бот (aiogram) ↔ app/services (правила) ↔ app/repo (SQL)
                                     app/core (агенты, расчёты, безопасность)
Mini App ↔ API (FastAPI) —————→ SQLite
Панель   ↔ API (admin)
Планировщик внутри процесса бота → утренние прогнозы, рассылки, гороскопы
```

Процессов **ровно два** — бот и API. Это принципиально: планировщик, утренние прогнозы и рассылки живут только в процессе бота, поэтому второй экземпляр бота недопустим (отправил бы каждое сообщение дважды). API масштабируется свободно, но держится один воркер — лимитеры in-process (см. ниже).

Слои:

- `app/bot/` — `aiogram`: онбординг, чаты, Таро, практики, профиль, оплата.
- `app/core/` — агенты (`agents/specs.py`, `agents/runtime.py`), расчёты (`astro.py`, `tarot.py`, `matrix.py`), память (`memory.py`), скиллы (`skills.py`), безопасность (`safety.py`, `flood.py`), персоны (`personas.py`), устойчивый сид (`stable.py`).
- `app/api/` — FastAPI: роутеры `today`, `chat`, `tarot`, `chart`, `practices`, `diary`, `shop`, `share`, `profile`, `admin`, `webhooks`, аутентификация по подписи Telegram.

- `app/services/` — правила продукта: `billing` (оплата/выдача), `limits`, `chat`, `practices`, `horoscopes`, `broadcast`, `referrals`, `invoices`, `scheduler`, `telegram`, `analytics`.
- `app/repo/` — SQL-слой (`users`, `payments`, `orders`, `plans`, `products`, `content`, `memory`, `events`...).
- `app/data/` — схема и миграции, сид каталога (`seed.py`).
- `app/pdfgen/` — генератор PDF-разборов.
- `admin/` — админ-панель (статический фронтенд к `/api/admin`).
- `web/` — публичный лендинг, `robots.txt`, `sitemap.xml`.
- `scripts/` — `selfcheck`, генерация промокодов и PDF, `healthcheck`, `seed`.
- `tests/` — `pytest`-наборы по подсистемам.
- `infra/` — `Dockerfile`, `docker-compose`, `Caddyfile`.
- `load/` — нагрузочные скрипты (`locust`, `simulate`).

Быстрый старт

```
cd oracle
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env          # вписать BOT_TOKEN, ADMIN_ID, WEBAPP_URL
python -m app.bot.main         # бот
uvicorn app.api.main:app --port 8080  # API / Mini App
```

Бот работает сразу:

- **без ключей** — офлайн-режим: шаблонные мистические трактовки на реальных данных карты, Таро и Матрицы;
- **с ключом Claude или OpenAI** (любым) — полный агентный режим.

Проверка проекта одной командой: `python -m scripts.selfcheck`.

Локальный просмотр Mini App: `http://localhost:8080/?dev_user=<твой_tg_id>` при `DEV_MODE=1`.

LLM-слой

Провайдеры (`LLM_PROVIDER=auto`) выстраиваются в **цепочку отказоустойчивости**:

`custom` → `anthropic` → `openai`. Упал основной — агент сам уходит на резервный; в конце — офлайн-режим. `CUSTOM_LLM_BASE_URL` подключает любой OpenAI-совместимый сервер.

Ключевые решения слоя:

- **Потоковый ответ обязателен** для OpenAI-совместимых серверов (`stream=True`): многие прокси и локальные раннеры отвечают SSE независимо от запроса, и при обычном разборе `content` приходит пустым.
- Размышления `reasoning`-моделей (`reasoning_content`) в ответ не попадают.
- Если модель не умеет `tool-use`, включается `pre-tool` режим: скиллы выполняются заранее и подкладываются в контекст.

- **Защита от всплеска:** семафор + rate-limit на вызовы (настраивается), чтобы сотни корутин не били в API одновременно (иначе 429-шторм и мгновенный уход в офлайн).
- **Таймауты:** на вызов и общий бюджет времени на запрос — мёртвый провайдер не держит ответ минутами.
- **Учёт себестоимости:** каждый вызов пишет `cost_usd` по `purpose` (`answer/forecast/memory/horoscope`) — это база юнит-экономики в `docs/MONETIZATION.md`.

Память

`app/core/memory.py` — долгая память: факты дедуплицируются **по смыслу**, ищутся под текущий вопрос и собираются в сводку. Поиск по словам; при флаге `semantic_memory` — по эмбедингам. Глубина памяти зависит от тарифа.

Безопасность

- **Кризисный протокол** (`core/safety.py`): сообщения о суициде, самоповреждении и насилии не доходят до модели — решение принимает код, а не промпт. Пользовательница получает поддержку и контакты помощи, лимит не тратится. Контакты правятся в панели (`safety.helplines`, по странам).
- **Антифлуд и лимиты:** rate-limiter исходящих Telegram (`core/flood.py`), дроссель запросов, лимиты вопросов по тарифу (атомарно для денежных путей).
- **Подпись Telegram initData** проверяется со сроком жизни 24 часа.
- **Вебхуки платёжек:** подпись по сырому телу + журнал `webhook_events` — повторная доставка никогда не выдаёт товар дважды (идемпотентность).
- **Деньги атомарны:** оплата → выдача в одной транзакции; повторный платёж не начисляет дни дважды.
- **Удаление данных:** `/delete_me` анонимизирует аккаунт (имя, дату рождения, карту, дневник, память, переписку); финансовый след остаётся по закону.
- **Устойчивый сид** (`core/stable.py`): встроенный `hash()` рандомизируется при каждом запуске Python — из-за этого карта дня и балл пары менялись после рестарта. Сид зафиксирован.
- `DEV_MODE=1` пускает в API любого по `?dev_user=<id>` без подписи — на боевом сервере флаг выключен.

Нагрузка и масштабирование

Проект проектировался под запуск с наплывом порядка 10 000 пользователей. Ориентиры:

- p95 ответа бота < 10 с, p99 < 30 с (включая LLM-цепочку и фолбэк).
- API (Mini App) p95 < 400 мс, p99 < 1,5 с, кроме генераций.
- Ошибки 5xx < 0,1%; ноль ошибок в денежных путях; аптайм 99,5%.
- Утренний прогноз до 09:30 по локальному времени пользователя.
- Рассылки: каждый таргет ровно один раз, вся доставка за час от слота.

Инженерные решения под эти цели:

- **Параллельная генерация** утренних прогнозов (семафор, предгенерация ночью, офлайн-шаблон при перегрузке) — последовательный проход по тысячам пользователей не укладывался бы в окно.
- **Пагинация аудитории** в рассылках (`ORDER BY tg_id`) — без неё хвост таймзоны никогда не получает прогноз.
- **Сериализация миграций на старте** — бот и API стартуют вместе, иначе второй процесс падает на втором `ALTER` .
- **Один воркер API**: лимитеры (rate-limit запросов, бакет Telegram) живут в памяти процесса; несколько воркеров дали бы каждому свой запас. Синхронизация появится вместе с переходом на PostgreSQL + Redis при росте до 30–50k пользователей.
- **Ретеншен событий** (`events` , `llm_usage`) — аналитика вне hot-path, purge 90–180 дней.

Тесты и проверка

- `pytest tests/ -q` — 250 автотестов по подсистемам: биллинг, лимиты, рассылки, планировщик, FSM-онбординг, безопасность, PDF, LLM-фолбэк без сети, API, миграции, гонки.
- `python -m scripts.selfcheck` — синтаксис, импорты, конфиг, деньги, лимиты, кризисный протокол, PDF, LLM-проб.
- CI (GitHub Actions) — lint, pytest + selfcheck на каждый PR.

Дорожная карта инженерных улучшений

Этапные цели (аудит, нагрузочные работы, доработки ядра) ведутся отдельным трекером в `.omc/ultragoal/` . В документацию проекта они не входят — это операционная работа, а не описание продукта. Актуальные планы деплоя и запуска — в `docs/OPERATIONS.md` .

Дизайн и бренд

Оракул — Дизайн-спецификация продукта

Статус: v2.0 (Astral Midnight · chat-first) · **Формат:** живой документ, ведётся вместе с кодом **Платформы:** ПК (Mini App в Telegram Desktop + Web) · Мобайл (iOS/Android) **Аудитория:** русскоязычные женщины 25–45, личный AI-астролог **Реализация:** `miniapp/styles.css` + `miniapp/app.js` — каждый токен и компонент ниже имеет прямое отражение в коде.

Документ описывает **полный продуктовый дизайн**: от дизайн-системы и визуального языка до архитектуры фронтенда и состояний. Это не «макет», это рабочая договорённость между дизайном и кодом.

0. Резюме одной страницей

Продукт — личный AI-астролог в концепции **chat-first**. Не «бот-говорилка», а проводник: честные расчёты (Swiss Ephemeris, Таро, Матрица Судьбы), которые ИИ трактует на живом русском. **Чат — главный инструмент**: экраны — лёгкая рамка вокруг диалога с агентом, а функции (расклад Таро, натальная карта, лунная неделя) живут кнопками прямо в диалоге.

Визуальная идея — «Astral Midnight». Тёмно-индиго космос с тёплым золотом и аметистом, glassmorphism, звёздное небо, 3D-переворот таро-карт. Премиум строится на калибровке: глубокий ночной фон, благородное золото, аметистовая подсветка, редакционная типографика, воздух. Мистика — в **точности расчёта**, а не в картинках.

Три правила больших данных (отвечают на боль про «супер-длинную страницу»):

1. **Одно главное число на экран** — глаз ловит ядро, остальное многоуровнево.
 2. **Прогрессивное раскрытие** — заголовок-цифра → раскрыть → детали.
 3. **Функции — в диалоге** — нет простыни экранов, есть чипсы и виджеты в чате.
-

1. Принципы (не обсуждаются)

1. **Честность расчёта = достоинство**. Дома, градусы, аспекты, фазы подаём как точные данные. Чем точнее цифра — тем «дороже» выглядит. Декор не заменяет расчёт.
2. **Два акцента**. Золото — для смысла и чисел (крупно, точками и линейками, никогда заливкой); аметист — для магии и подсветки. Акценты — точки свечения, не плашки.

3. **Воздух — это роскошь.** Больше пустоты, чем контента на экран управления.
4. **Продукт не навязывает.** Лимиты и монетизация — мягкие карточки, не «дверь в лицо».
5. **Данные сдаются в сетку, а не в абзац.** Числа живут в колонках, не в прозе.
6. **Чат — приоритет.** Интерфейс не конкурирует с диалогом: рамка тихая, агент громкий.

2. Визуальный язык

2.1 Дизайн-токены (тёмная тема — единственная на старте)

Палитра **Astral Midnight**: глубокий космос (`#08070f`), индиго-поверхности, тёплое золото и аметист, полупрозрачное стекло. Все значения — реальные CSS-переменные `miniapp/styles.css`.

Токен	Значение	Назначение
<code>--bg-0</code>	<code>#08070f</code>	основной фон, «ночной космос»
<code>--bg-1</code>	<code>#0e0d1e</code>	подложка шапок, градиенты
<code>--bg-2</code>	<code>#151330</code>	модалы, вложенные блоки
<code>--glass</code>	<code>rgba(24,22,48,.55)</code>	glass-карточки + <code>backdrop-blur</code>
<code>--glass-strong</code>	<code>rgba(28,26,56,.72)</code>	пузыри агента, навбар
<code>--gold</code>	<code>#e6c178</code>	золото-якорь: числа, заголовки, кайма
<code>--gold-bright</code>	<code>#ffd98f</code>	светлое золото: активное, имена
<code>--gold-deep</code>	<code>#b98946</code>	тень золота, градиенты кнопок
<code>--gold-glow</code>	<code>rgba(230,193,120,.35)</code>	золотое свечение
<code>--violet</code>	<code>#a78bfa</code>	аметист: подсветка, магия
<code>--violet-glow</code>	<code>rgba(167,139,250,.35)</code>	аметистовое свечение
<code>--text</code>	<code>#f4f1ff</code>	основной текст, светлый
<code>--text-dim</code>	<code>#a49cc8</code>	подписи, meta ($\geq 4.5:1$ на фоне)
<code>--text-faint</code>	<code>#6f6890</code>	декоративный/необязательный
<code>--border</code>	<code>rgba(190,170,255,.13)</code>	hairline-границы
<code>--border-strong</code>	<code>rgba(230,193,120,.42)</code>	золотая кайма акцентов
радиусы	<code>14 / 20 / 28px</code>	<code>--r-s / --r-m / --r-l</code>
тень карт	<code>0 18px 50px -18px rgba(0,0,0,.85)</code>	<code>--sh-card</code>
тень золота	<code>0 0 0 1px gold·06 + 0 18px 50px -22px gold·18</code>	<code>--sh-gold</code>

Токен	Значение	Назначение
easing	<code>cubic-bezier(0.22, 1, 0.36, 1)</code>	<code>--ease</code> , все движения

Правило золота (критично): золото — акцент крупного и светящегося. Мелкий текст — `--text` / `--text-dim` (светлый), НИКОГДА золотом. Золото применяется к: дисплейным числам и именам, каймам (`border-strong`), фокус-кольцу, свечению (`glow`), иконкам. Аметист — к подсветкам (`violet-glow`), коніс-кайме кольца агентов, акцентам магии.

Контраст выдержан: текст $\geq 4.5:1$, крупный текст и иконки $\geq 3:1$.

2.2 Типографика — двигатель премиальности

Роль	Семейство	Замечание
Дисплей (заголовки, имена, числа)	Cinzel (500/600/700, кириллица)	античный serif, капитель, буквенная разрядка
Тело	Plus Jakarta Sans (400–800)	гуманистический гротеск, читаемость
Числа/градусы	табулярные (<code>font-variant-numeric: tabular-nums</code>)	«банковские» цифры без моношрифта
Meta/eyebrow	капс + <code>letter-spacing 2px</code> , размер 10–12px	подписи дат, секций

Подключение — **самохостинг** под строгий прод-CSP (`style-src 'self'`): woff2 лежат в `miniapp/fonts/` (Cinzel + Plus Jakarta Sans, кириллица и unicode-range сохранены из Google Fonts), подключены через `oracle-fonts.css`; ассеты с cache-busting `?v=1`. Заголовки секций — Cinzel капителью с буквенным трекингом, «сегодня» — gradient-text золото→аметист.

2.3 «Один решительный флориш»

Звёздное небо (`.starfield`) — единственный повторяющийся фон-приём: фиксированный слой с радиальными градиентами (индиго-верх, аметист-низ) и дрейфующими звёздными точками (золотые, белые, аметистовые), две слои анимации `starDrift 90/140s` в разные стороны. Поверх — glass-интерфейс. Никаких россыпей эффектов сверх этого.

2.4 Графика

Дефолт — эмодзи-глифы агентов и карт (данные приходят с бэкенда) + стилизованные орбы-градиенты. Реалистичные изображения — только точечно. Иконки — `line-style`; аватары агентов — радиальные градиенты с каймой accent-цвета (`::before mask-composite`). Никаких wireframe-боксов.

3. Компонентная система

3.1 Базовые

- **Glass-карточка** (`.glass`) — `rgba`-заливка + `backdrop-filter: blur(10px)` + `border 1px` + `border-radius 20px` + мягкая тень `--sh-card`. Акцентная — `.glow-border` с градиентной каймой золото→аметист (`mask-composite xor`). Наведение — подъём и уточнение каймы.
- **Button** — `.btn-primary` (золотой градиент `gold-bright-gold-deep`, тёмный текст, свечение `--sh-gold`) и `.btn-ghost` (`hairline` + светлый текст). Высота $\geq 44\text{px}$, точная область $\geq 44\text{px}$.
- **Input / Textarea** (`.ipt`) — тёмная подложка `rgba(10,9,22,.6)`, `hairline`-кайма, фокус: золотая кайма + кольцо `--gold-glow` ($\geq 3:1$). Плейсхолдер — `--text-faint`.
- **Feature-чипс** (`.chip`) — капсула: эмодзи + текст, `hairline`, `hover` → `accent`-кайма. Живёт в хабе агентов и в полосе фич чата. Виды: `chip` (действие), `suggest-chips` (подсказки).
- **Spread-ячейка** (`.spread-cell`) — выбор схемы расклада в чат-виджете; `.premium` — золотой текст + ; `.sel` — золотая кайма + свечение.
- **Card-day** — карточка «карта дня»: вертикальная мини-карта с золотой каймой.
- **Stat** — ячейка статистики профиля: табулярное число золотом + подпись.
- **Result-card** — карточка результата (расклад/разбор): заголовок `serif` + `meta`, `hover` — сдвиг.
- **Modal** (`.modal-overlay` / `.modal`) — затемнение `blur(6px)` + стеклянная панель, появление `popIn`. Используется для чтения расклада и разбора.
- **Skeleton** — shimmer-пульс в тон фона (не спиннер).
- **Badge-dot** — золотая точка-уведомление с `glow`.

3.2 Компоненты chat-first

- **Кольцо агентов** (`agent-dock`) — на «Сегодня»: орбы-аватары с `conic`-каймой золото→аметист (`mask-composite: xor`), `hover` — подъём + аметистовое свечение.
- **Карточка агента** (хаб): аватар с `accent`-каймой, имя `serif`, роль `accent`-цветом, превью `last_text` (`ellipsis`) + **полоса фич-чипсов**.
- **Чат-виджет** (`.chat-widget`) — системное сообщение-форма внутри ленты: заголовок `serif` золотом + содержимое фичи (расклад, натальная карта, форма совместимости).
- **Пузыри сообщений** — клиентка: золотой градиент + тёмный текст, скругление `18px 18px 5px 18px`; агент: `--glass-strong` + `hairline`, `18px 18px 18px 5px`.
- **Таро-карта** (`.tarot-card`) — 88×136 , 3D-переворот 700ms (см. §5.1); рубашка — индиго-градиент + золотая кайма + мотив «ORACLE».
- **Hero «Сегодня»** (`.hero-orb`) — высокая карта с плавающей золото-аметистовой орбой (`orbFloat 8s`), дата капсом, приветствие `serif`, строка Луны.

3.3 Паттерн «цифра → раскрытие»

Универсальный решатель «длинной страницы»:

[Солнце · Дом VII · 12°] ← строка-сводка, кликабельна
▼ раскрыть
[детали: управитель, аспекты, ← аккордеон
трактовка в 1 абзаце]

В chat-first применяется в виджетах (карта, расклад) и Профиле. Правило: **в свёрнутом виде одно число/одна строка; раскрытие — по запросу.**

3.4 Паттерн «одно главное число»

На «Сегодня» — ядро дня (прогноз + карта дня) крупно вверху. Всё остальное — второстепенные виджеты ниже. Глаз не сканирует простыню — он ловит ядро.

4. Навигация и информационная архитектура

4 экрана (нижний таб-бар `app-nav`, максимум 480px):

Экран	Главная цель	Ключевые элементы
Сегодня	ядро дня за 5 сек, статичная база	hero-орба, прогноз, карта дня, кольцо агентов
Агенты	выбор собеседника	карточки агентов + фичи-чипсы
Чат	главный инструмент	шапка, полоса фич, лента, композер, чат-виджеты
Профиль	хранилище результатов	статистика, натальная карта, расклады, разборы, память

Функции бывших отдельных экранов (Таро, Карта, Практики, Дневник, Лавка) живут **фичами в диалоге** и результатами в Профиле. Единый якорь-флориш — звёздное небо и золото-аметистовое свечение в одной тональности на всех экранах.

Запоминание: анкор-экран «Сегодня» — узнаваемое «лицо» продукта (hero + карта дня); остальные экраны — спокойная рамка, чтобы диалог оставался главным.

5. Состояния и ошибки (часть дизайн-системы)

У каждого виджета/списка 5 состояний: **loading · empty · error · data · edge**.

- **Loading** — скелетон в тон фона (shimmer-пульс) или loader-ring; НЕ спиннер-вращалка.
- **Empty** — фирменная пустота: «Здесь пока тихо», 1 строка подсказки, СТА (например, «Собери карту у Астролога — прямо в чате»).
- **Error** — что случилось + что делать + retry с бэкоффом 2/4/8с, затем «Связаться с поддержкой» + ID.
- **Data** — наполнение по паттернам §3.
- **Edge** — перенос строк, усечение с многоточием, `overflow-wrap`, проверка RTL.

Reduced motion: `prefers-reduced-motion` → убрать трансляции/перевороты, оставить кроссфейды. Фокус: `:focus-visible` с золотым кольцом, контраст $\geq 3:1$.

6. Микроинтерфейс и тон

- **Переходы между экранами** — кроссфейд 200–300ms, без слайдов-спрингов.
- **Чат-виджеты** — появление `msgIn` (fade + подъём 8px, 350ms), staged для пузырей.
- **3D-переворот Таро** — единственное место, где допускается «театр» (700ms); с `prefers-reduced-motion` → мягкий кроссфейд.
- **Наведение на карточку** — подъём 1-2px + уточнение каймы, 150ms; чипс — подсветка accent.
- Звук — нет (астрогештальт, без аудио-декора).

Тон текста: тёплый, таинственный, но без манипуляций. «Время рождения неизвестно» подаётся как доверие, а не как укор.

7. Темы

Тёмная «Astral Midnight» — единственная на старте (§2.1, CSS-переменные в `:root`). Светлая тема («пергамент») — в плане, из тех же токенов через `data-theme`, без дублирования компонентов.

8. Архитектура и стек (фронтенд)

8.1 Стек (фактический)

Vanilla JS + CSS, без сборки. `miniapp/` отдаётся FastAPI как статика (`/static` → `miniapp/`, `StaticFiles`): `index.html` + `styles.css` + `app.js`. Одиночный объект `window.app` с методами-экранами и реестром фич. Причины:

- Гринфилд уже реализован: 4 экрана + чат-виджеты на vanilla — просто и предсказуемо.
- Нет билд-стадии — нет гонки деплоя ассетов и npm-стадии в Docker.
- `@twa-dev/sdk` — **НЕ нужен**. Авторизация серверная: `X-Init-Data` (HMAC по `bot_token`). Клиенту нужен лишь `Telegram.WebApp.initData`, который Telegram впрыскивает сам.
- Если интерфейс вырастет (полное SVG-колесо, октаграмма, спидометр) — кандидаты: нативный SVG; фреймворк подключать только при явной необходимости.

8.2 Сборка и хостинг

Без сборки: `index.html` подключает `/static/fonts/oracle-fonts.css`, `/static/styles.css` и `/static/app.js` (с `?v=1`); шрифты — локальные woff2 из `miniapp/fonts/` (прод-CSP не

пускает сторонние стили); `telegram-web-app.js` — официальный скрипт Telegram. Корень FastAPI плотно занят роутами — ассеты под `/static/`.

8.3 Структура

```
miniapp/  
├─ index.html      # каркас, telegram-web-app.js, .starfield, cache-busting ?v=1  
├─ styles.css      # дизайн-система Astral Midnight (токены + компоненты)  
├─ fonts/          # самохостинг шрифтов: oracle-fonts.css + woff2 (Cinzel, PJS)  
└─ app.js          # chat-first приложение: api-клиент, реестр FEATURES,  
                   # экраны (home/hub/chat/profile), чат-виджеты;  
                   # 0 inline-хендлеров — делегирование data-act + rich()-  
экранирование
```

8.4 Данные из API

- Экраны ↔ эндпоинты FastAPI, формат JSON; ошибки — `{detail}`.
- **Свёрнутые данные приходят свёрнутыми** (§3.3) — детали lazy-запросом.
- Клиент — тонкая обёртка `fetch` с заголовком `X-Init-Data` (§2 FRONTEND_TZ).

8.5 Бэкенд-стык (уже есть)

FastAPI + встроенный веб; монетизация: триал 30дн → «Искра»; лимиты «пламя» + экстренный вопрос за Кристаллы ✦; 6 агентов (oracle/tarot/astro/coach/numero/keeper); честные расчёты (Swiss Ephemeris, Таро, Матрица Судьбы) — душа, которую интерфейс бережно показывает.

9. Дорожная карта сборки

1. **Реализовано:** токены и палитра Astral Midnight; типографика (Cinzel × Plus Jakarta Sans); каркас (шапка, таб-бар, звёздное небо); экраны «Сегодня», «Агенты», «Чат», «Профиль»; чат-виджеты (Таро с 3D-переворотом, натальная карта, луна, матрица, совместимость, прогноз, история).
2. **Фаза состояний:** полный прогон loading/empty/error/edge у всех виджетов + retry-бэкофф.
3. **Фаза премиум:** пейволл-карточка в чате, лавка, «сбылось», сторис, практики/дневник фичами, полное SVG-колесо, октаграмма, спидометр, стрик.
4. **Фаза polish:** reduced-motion, фокус, микроанимации, аудит контраста.

Критерий «выглядит дорого»: одновременно выполнены (а) одно главное число, (б) воздух, (в) золото только точками и свечением, (г) табулярные цифры, (д) ни одной золотой заливки и ни одной «дешёвой» звёздной картинкой (только `.starfield`).

10. Глоссарий

- **Пламя** — геймифицированный лимит вопросов; мягкая карточка, не блок.
- **Кристаллы ✦** — валюта для экстренных вопросов и покупок.

- **Искра** — бесплатный план «карта дня + вопрос/нед».
- **Hairline** — граница 1px высокой точности, «золото точками» в линиях.
- **Glassmorphism** — полупрозрачные поверхности `backdrop-blur` над звёздным небом.
- **Фича-чипс** — кнопка-функция агента в диалоге (см. FRONTEND_TZ §4.3).
- **Astral Midnight** — дизайн-система: космос + золото + аметист.

Оракул — T3 и план реализации фронтенда (Mini App · chat-first)

Статус: v2.0 (chat-first) · **Верхнеуровневый документ:** [DESIGN_SPEC.md](#) (дизайн-система Astral Midnight, токены, компоненты) **Этот документ — техзадание:** построчно привязан к реальному API бэкенда и к фактической реализации `miniapp/app.js` + `miniapp/styles.css`. Каждый экран, элемент и состояние опирается на реальный контракт, а не на пожелание.

0. Цель и границы

Цель — Telegram Mini App «Оракул» в концепции **chat-first**: четыре экрана (Сегодня · Агенты · Чат · Профиль) и **функции, живущие кнопками прямо в диалоге** с ИИ-агентом. Чат — главный инструмент; экраны — лёгкая рамка вокруг него.

Границы (честно, чего НЕ делаем): - Пуш-уведомления — на бэкенде только флаг `morning_push` и время; доставки фронту нет. Реализуем UI и хранение настройки, механику — на боте. - «Книга судьбы» как отдельная сущность не выделена в данных (это дневник + память). Экран «Дневник» в chat-first не нужен: запись ведётся в боте, а в Mini App итоги подводит фича «Итог месяца» у Хранителя дневника (keeper). -

Онбординг (дата рождения/время/город/имя Оракула/onboarded) происходит В БОТЕ (FSM `app/bot/onboarding.py`) — в API нет эндпоинта записи `birth_date / onboarded`. Mini App лишь встречает `onboarded=0` мягкой карточкой «Заверши настройку в боте» + СТА. - **Исключение — натальная карта:** `birth_date` пишется только в боте, но **время и город рождения клиента может добавить/уточнить прямо в чате** через `POST /api/chart {birth_time, birth_city}` (см. §4.5). Дата рождения без неё — 400. - Профиль меняет только `oracle_name/persona/morning_push/tz/goal` через `POST /api/profile` — никогда не пишет дату рождения и `onboarded`. - Отчёт `monthly` в REPORTS не подтверждён; подтверждены kinds: **natal/matrix/synastry/solar/career**. Показываем только те kinds, что реально возвращает `GET /api/reports`. - Отдельных экранов Таро / Карты / Практик / Дневника / Лавки НЕТ — их функции раскладываются фичами в чате и результатами в Профиле.

1. Базис (решения из DESIGN_SPEC)

Аспект	Решение
Стек	Vanilla JS (<code>miniapp/app.js</code>) + CSS (<code>miniapp/styles.css</code>), без сборки — FastAPI отдаёт <code>miniapp/</code> как статику
Токены / темы	

Аспект	Решение
	DESIGN_SPEC §2 — тёмная тема «Astral Midnight»: космос #08070f, золото #e6c178, аметист #a78bfa, glassmorphism
Типографика	Cinzel (дисплей, кириллица) × Plus Jakarta Sans (тело), табулярные цифры
Ключевые паттерны	chat-first: реестр фич-чипсов агента → виджет в диалоге; «одно главное число» на «Сегодня»
Навигация	4 экрана: Сегодня · Агенты · Чат · Профиль (нижний таб-бар)
Каркас	#app-root ≤ 480px, sticky-шапка, backdrop-filter -навбар, .screen — свой скролл-контейнер

2. Архитектура и интеграция

2.1 Авторизация (клиент ↔ FM-контракт)

Авторизация — **только заголовок X-Init-Data** (не cookie, не Bearer). Клиент обязан слать его с каждым запросом. Реализация — тонкая обёртка `api(path, opts)` в `app.js`:

```
async function api(path, opts = {}) {
  const headers = { 'Content-Type': 'application/json', ...opts.headers };
  const initData = window.Telegram && window.Telegram.WebApp.initData;
  if (initData) headers['X-Init-Data'] = initData;
  // dev-режим: ?dev_user=<id> (только когда settings.dev_mode=1, иначе 401)
  ...
  if (!res.ok) { const err = new Error(detail || 'Связь прервалась '); err.status =
res.status; throw err; }
  return body;
}
```

Критично: - **ВСЕ запросы** (кроме публичных `/health`, `/faq`) несут `X-Init-Data`. - Payload, который ДОЛЖНА протащить авторизация: `tg_id` из `Telegram.WebApp.initData.user.id`. - Фронт НЕ расшифровывает `initData` сам — бэкэнд валидирует (HMAC-SHA256 по `bot_token`, `auth_date` ≤ 24 ч). Фронт лишь пробрасывает строку и обрабатывает 401/404. - Ошибка бэкенда — `{"detail": "<человечный текст с >"}`; фронт показывает её как есть.

2.2 Коды ошибок → состояние экрана (единый маппинг)

Всё приложение оборачивает ответы в один HTTP-слой. Семантика кодов от бэкенда:

Код	Смысл	Что делает фронт
200	ок	отрисовать успех
401	нет юзера (подпись не прошла)	показать «Открой бота и нажми /start», кнопка-переход в Telegram
403	доступ приостановлен (#блок)	экран «доступ приостановлен»

Код	Смысл	Что делает фронт
404	юзер/ресурс не найден	empty-состояние + «как начать»
402	подписка истекла / разбор не куплен	пейволл-карточка (см. §6.2), СТА «Продлить»
429	лимит: <code>rate_limit</code> ИЛИ <code>limit_reached</code> (вопросы/чаты/расклады)	карточка лимита «пламя»/Кристаллы + <code>retry</code> , если это <code>rate limit</code>
500	сервер	«Связь со звёздами прервалась...» + <code>retry</code> с бэкоффом 2/4/8с + «Связаться» + ID
503	оплата недоступна	мягкая заглушка + «попробуй позже»

Формат ошибки — `{"detail": "..."};` сообщение приходит из бэкенда (`DENY_TEXT` в `app/api/routers/chat.py`), фронт не придумывает свой текст.

2.3 Стартовый взлёт: `/api/me` — «золотой старт»

`GET /api/me` отдаёт почти всё в одной пачке. Фронт при загрузке делает его ОДИН раз и раскладывает в стор:

```
user { tg_id, name, username, oracle_name, persona, tz, birth_date, birth_city,
      birth_time_known, onboarded, sun, ascendant, chart_mode, planets }
crystals, sub_active, sub_days_left, plan, allowance{...}
questions_left, questions_total, memories, diary_streak, morning_push
entitlements, reports, agents, flags, webapp_url
```

Для онбординга ключевой — `onboarded`. Если `onboarded=0` → гнать поток §6.1.

2.4 Кэш и фиче-флаги

- Статику кормит `Cache-Control` бэкенда: ассеты `public`, `max-age=3600`, всё остальное `no-cache`. Фронт не трогает заголовки.
- **Фиче-флаги:** `me.flags` — `map code=bool`. Ключевые: `share_cards` (показывать блок «в сторис»), `web_payments` (web-оплата). Код обязан дефолтить отсутствующие флаги к `false`.

2.5 Ограничения Telegram WebApp (Mobile Architect)

- **WebView — не браузер ПК.** Базовая раскладка — мобильная (каркас $\leq 480\text{px}$), десктоп — прогрессивное расширение через `media-запросы`.
- **Haptic:** `Telegram.WebApp.HapticFeedback.impactOccurred("light"|"medium")` — на Таро переворот, смену вкладки, покупку. **Безопасно** оборачивать в `try` (не все клиенты).
- **Тема Telegram** не навязываем — у нас своя тёмная `data-theme`; `themeParams` НЕ источник токенов. `colorScheme` только как подсказка.
- **Геометрия:** следим за `viewportStableHeight` для раскладки таб-бара.
- **Скролл-контейнер:** свой (`.screen`, `.chat-messages`), не `body` — для таб-бара и шапки.
- Эмодзи в данных (агенты, карты) — рендер как есть, подмена иконок не нужна.

3. Стор-модель (единый источник на фронте)

Реализация — одиночный объект `window.app` (без фреймворка):

```
const app = {
  me: null, agents: [], today: null, spreads: null, // данные с бэкенда
  view: 'home', // home | hub | profile
  chat: { key: null, spec: null, messages: [], pending: null, busy: false },
}
```

Правила: - разовый `/me` на бут (`app.boot()`); точечные апдейты после мутаций (`/chat`, `/tarot/draw`, `/chart`, `/compat/full`) — мерджем в локальный стор, не перезагрузка `/me`. - `chat.pending` — **активный чат-виджет**: объект `{kind, ...}` (`tarot-pick`, `tarot-cards`, `chart`, `compat`, `moon`, `matrix`, `today`, `history`). Пока `pending` жив, он рисуется как системное сообщение агента в ленте (§4.3).

4. Поэкранное ТЗ (chat-first)

Каждый экран: **назначение** → **иерархия** → **интеракции** → **API-контракт** → **граничные случаи**. У каждого виджета 5 состояний (`loading` / `empty` / `error` / `data` / `edge`) — §7.

4.0 Онбординг (сквозной, см. §6.1)

Ведёт бот (FSM); Mini App при `onboarded=0` показывает карточку «Заверши настройку в боте» + СТА. См. §6.1.

4.1 Сегодня (статичная база)

Назначение: ядро дня за 5 секунд; ритуал открытия; главная страница Mini App. **Никакой интерактивной функции здесь нет** — всё, что требует вопроса, живёт в чате.

Иерархия (сверху вниз, `renderHome`): 1. **Hero** (`hero-orb`): дата (`hero-date`), приветствие по имени, строка Луны `{moon.name} · {moon.day}`-й день из `today.moon`. Плавающая «орба» — золото-аметистовый градиент со свечением. 2. **Прогноз на сегодня**: glass-карточка с `today.forecast` (2–4 строки). 3. **Карта дня** (`card-day`): `today.card {emoji, name, meaning}` — маленькая «карта» в рамке. 4. **Твои агенты** (`agent-dock`): **кольцо агентов** — аватары-орбы с `conic`-каймой золото→аметист; тап по орбу → `openChat(code)`. Подпись-подсказка: «Открой агента — задай вопрос или используй его функцию прямо в чате».

Состояния: - `onboarded=0` → карточка «Заверши настройку в боте» (§6.1). - `/today` зависит от LLM и может таймаутить → скелетон (`skeleton`) + повторный `retry`-бэкофф;

если день уже грузился — показать из стор. - 429 на `/today` → пейволл-лимит, НЕ обнулять уже показанный прогноз.

API: `GET /api/today` → `{forecast, card, moon, sun_season, day}` · `GET /api/agents` (для кольца) · `GET /api/me`.

4.2 Агенты (хаб)

Назначение: выбор собеседника. Карточка агента несёт **фичи-чипсы** — прямое окно в «функции в диалоге».

Иерархия (`renderHub`): 1. Заголовок: «Твой Оракул» + подпись «Чат — главный инструмент. Выбери агента: задай вопрос или нажми его функцию.» 2. Список (`GET /api/agents`): карточка на агента — `avatar(emoji, accent-кайма)`, `name`, `title`, `last_text` (превью последнего сообщения или `greeting`). 3. **Фичи-чипсы агента** (первые 4 из реестра §4.3): тап по чипсу → открывает чат с агентом и сразу запускает фичу (`openChat(code, () => app.<handler>())`). `event.stopPropagation()` — чтобы чипс не открывал просто чат.

API: `GET /api/agents` (СПИСОК: `code`, `name`, `emoji`, `title`, `tagline`, `accent`, `greeting`, `suggestions`, `last_text`, `last_at`, `msg_count`).

4.3 Чат — главный инструмент (каркас + реестр фич)

Назначение: вся работа продукта. Вопросы — вводом, функции — кнопками над лентой.

Иерархия (`renderChat`): 1. Шапка (`chat-head`): кнопка «<» назад, аватар+имя+ `title` агента (`accent-кайма`), кнопка «У» очистить тред (`DELETE /api/chat/{agent}` с `confirm`). 2. **Полоса фич** (`chat-features`): горизонтальный скролл чипсов из реестра `FEATURES[agent.code]` — тап запускает виджет прямо в диалоге. 3. Лента (`chat-messages`): сообщения `.msg.user` / `.msg.assistant` + системный **чат-виджет** (`chat-widget`) для активной фичи + индикатор «печатает...» (`typing`). 4. Композер: поле ввода + кнопка «>» (`Enter` отправляет) + подсказки-чипсы (`agent.suggestions`).

Реестр фич (`FEATURES`) — единственное место, где задаётся набор кнопок агента:

Агент (code)	Фичи-чипсы	Обработчики
oracle (Лилит)	Расклад Таро · Прогноз дня · Натальная карта · Лунная неделя · Совместимость · Матрица	<code>featureTarot</code> , <code>featureToday</code> , <code>featureChart</code> , <code>featureMoon</code> , <code>featureCompat</code> , <code>featureMatrix</code>
tarot (Мадам Ленорман)	Расклад Таро · История	<code>featureTarot</code> , <code>featureTarotHistory</code>
astro (Урания)	Натальная карта · Небо сегодня · Лунная неделя · Совместимость	<code>featureChart</code> , <code>featureToday</code> , <code>featureMoon</code> , <code>featureCompat</code>
numero (Пифия)	Матрица Судьбы	<code>featureMatrix</code>

Агент (code)	Фичи-чипсы	Обработчики
coach (Ная)	Подобрать практику	chatPractice → вопрос агенту
keeper (Мнемо)	Итог месяца	chatMonthly → вопрос агенту

Два класса фич: - **Виджеты** (chat.pending с kind): инпут/форма/карты внутри системного сообщения — §4.4–4.6. - **Вопросы агенту**: chatPractice / chatMonthly — просто шлют текст в POST /api/chat/{agent} , скиллы агента сами решают, чем ответить.

Отправка (doSend): пуш своего сообщения в стор → POST /api/chat/{agent} body {text} → ответ агента {answer} пушится в ленту. Во время запроса — «печатает...» + блок повторной отправки (busy). 402/429 из detail показываются пузырьём агента (текст уже человеческий, от бэкенда).

API: GET /api/chat/{agent} → {agent: {code,name,emoji,title,tagline,accent,...}, thread_id, messages: [{role, text}]} · POST /api/chat/{agent} {text, allow_paid?} → {answer, questions_left, allowance} · DELETE /api/chat/{agent} (архив).

Движение: пузыри агента появляются staged (fade+slide); автоскролл вниз. Длинный тред → «загрузить ещё» (у бэкенда пагинации нет — грузим 60, кнопка).

4.4 ФИЧА: Таро в чате (вопрос → расклад → LLM)

Назначение: расклад начинается с вопроса клиентки; вопрос сохраняется в раскладе и передаётся в LLM-трактовку. Всё — внутри диалога таролога.

Флоу (featureTarot → doDraw → flipCard → doInterpret): 1. **Вопрос и схема** (pending.kind='tarot-pick'): системное сообщение «Давай сделаем расклад. Сначала выбери схему и напиши свой вопрос картам...»: - сетка раскладов GET /api/tarot/spreads ({code,title,positions,tier,emoji,...}), платные помечены ; - <textarea id="tarot-q"> — **формулировка вопроса к картам** (обязательна); - кнопка «Потянула карты ». 2. **Раздача (doDraw)**: POST /api/tarot/draw?spread={code} body {question: qv} → {reading_id, title, positions, cards, spread, thread_id} . Вопрос пушится в ленту как сообщение клиентки: «Мой вопрос к картам: ...». Виджет переходит в pending.kind='tarot-cards' : карты **лицом вниз** по позициям. 3. **Переверот (flipCard)**: тап по карте → .flipped (3D rotateY, \$5.1), Haptic light. Пока перевёрнуты не все — подсказка «Нажми на карты, чтобы перевернуть ». 4. **Трактовка (doInterpret)**: когда все перевёрнуты — кнопка «Что это значит для меня?» → POST /api/tarot/interpret/{reading_id} → {answer} ; ответ пушится в ленту. LLM-трактовка читает вопрос (бэкенд передаёт его в промпт) и разбирает по позициям.

История (featureTarotHistory): GET /api/tarot/history (последние 30) → карточки {question, spread, created_at, cards[0].emoji, answer} ; тап → модал со списком карт и трактовкой (openReading).

Состояния: - Без вопроса → локальный `alert` «Сформулируй свой вопрос картам». - 429 на `/draw` → пейволл-лимит (расклад и вопрос делят один лимит — бэкенд списывает атомарно). - `interpret` долгий (LLM) → «печатает...»; таймаут → продуктовый `retry`.

API: `GET /api/tarot/spreads` · `POST /api/tarot/draw?spread= (body {question})` · `POST /api/tarot/interpret/{id}` · `GET /api/tarot/history` · `POST /api/tarot/outcome/{id}` («сбылось» — в будущем, §5.5) · `GET /api/share/reading/{id}.png` (сторис).

4.5 ФИЧА: Натальная карта в чате → Профиль

Назначение: карта строится прямо в диалоге астролога (данные из онбординга или заполняются на месте), сохраняется и навсегда видна в Профиле.

Флоу (`featureChart` → `doBuildChart`): 1. `GET /api/chart` : - **карта есть** → виджет `chartHtml` : мини-колесо (`chart-wheel` , символ Солнца в центре, AC), строка «Солнце в {sign} · Асцендент {sign}», список планет {name, sign, house, retro} по линиям, пометка «✓ Сохранена в твоём профиле», кнопки «**Спросить про карту**» (`chatAsk` — вопрос агенту по карте, например про отношения) и «**В профиль**»; - **карты нет** (400 «карта ещё не построена») → виджет `chartForm` : подсказка «Дата рождения: {birth_date}. Уточни время и город — и я рассчитаю карту прямо здесь», поля `birth_time` («14:30», пусто = неизвестно) и `birth_city` , кнопка «Рассчитать». 2. **Постройка** (`doBuildChart`): `POST /api/chart` `body {birth_time: string|null, birth_city: string|null}` → сервер рассчитывает по Swiss Ephemeris, **сохраняет** (`users.update birth_city/birth_lat/birth_lon/tz/birth_time`) и возвращает {sun, ascendant, planets, houses, mode} . Виджет переходит в `chartHtml` . 3. **В Профиле** карта читается заново через `GET /api/chart` (§4.7) — единый источник.

Грани: без `birth_date` в БД → 400 «нет даты рождения — заполни её в боте» (показать карточку §6.1); время не в формате ЧЧ:ММ → 400 (валидирует бэкенд); повторный POST с пустыми полями при существующей карте → {cached: true} без пересчёта.

API: `GET /api/chart` → {mode, sun, ascendant, mc, planets[], houses[], aspects[], note, birth{date,time,city,time_known}} · `POST /api/chart` (body `ChartBuildIn`).

4.6 ФИЧИ-виджеты: Луна · Матрица · Прогноз · Совместимость

Все — системные сообщения-виджеты внутри чата (`pending.kind`):

- **Лунная неделя** (`featureMoon` , `kind='moon'`): `GET /api/moon/week` → список дней {date, weekday, day_num, name, emoji, day} строками `дд Пн · Имя — n-й день` .
- **Матрица Судьбы** (`featureMatrix` , `kind='matrix'`): `GET /api/matrix` → строки по ключам `personal/destiny/love/money` ({arcana, n}): «Личный · Аркан ...» . Подсказка: «Попроси агента разобрать твои арканы подробнее — просто напиши ему» .
- **Прогноз / Небо сегодня** (`featureToday` , `kind='today'`): `GET /api/today` → сообщение: дата + `forecast` + строка карты дня.

- **Совместимость** (`featureCompat` , `kind='compat'`): форма «Имя (необязательно)» + «Дата рождения партнёра · ГГГГ-ММ-ДД» → `POST /api/compat/full {partner_date, partner_name, save: true}` → `{answer}` (разбор Астрологом, расходует вопрос дня) → ответ пушится в ленту; вопрос клиентки сохраняется в тред астролога.

4.7 Профиль — хранилище результатов

Назначение: натальная карта, последние расклады, купленные разборы, воспоминания. Отсюда — снова спросить агента.

Иерархия (`renderProfile` + `loadProfileSections`): 1. **Статистика** (`stat-row`): подписка (∞/—), Кристаллы ✦, вопросы (`allowance.left`), стрик дневника (`diary_streak`). 2. **Твои данные:** дата рождения, время (или «не известно»), город — из `/api/me`. 3. **Натальная карта** (`#profile-chart`): `GET /api/chart` → колесо + Солнце/Асцендент + планеты + кнопки «**Спросить**» (`openChat('astro')`) и «**Разбор**» (`openReport('natal')`). Карты нет → пустая карточка «Собери её у Астролога — прямо в чате» + кнопка «Построить карту» → `openChat('astro')` с фичей. 4. **Последние расклады** (`#profile-tarot`): `GET /api/tarot/history`, первые 5 → карточки `{question, date, spread}`; тап → модал (карты + трактовка). Пусто → «Раскладов пока нет — зайди к Тарологу и задай вопрос картам.» 5. **Разборы** (`#profile-reports`): `GET /api/reports` → `ready[]` → карточки `{kind, title, period}`; тап → `GET /api/reports/{kind}` → модал с текстом. Пусто → «Разборов пока нет — они появляются в лавке.» 6. **Память** (`#profile-memories`): `me.memories` (до 8) → список «✦ факт». Пусто → «Я запомню о тебе важное, когда расскажешь.»

API: `GET /api/me` · `GET /api/chart` · `GET /api/tarot/history` · `GET /api/reports` · `GET /api/reports/{kind}`.

4.8 API-карта бэкенда (полный контракт, проверен по `app/api/routers/*`)

Чат и агенты — `app/api/routers/chat.py` `GET /api/agents` · `GET /api/chat` (лента Оракула) · `GET /api/chat/{agent}` · `POST /api/chat/{agent}` · `POST /api/ask` · `DELETE /api/chat/{agent}`

Таро — `app/api/routers/tarot.py` `GET /api/tarot/spreads` · `POST /api/tarot/draw?spread=&question=` · `POST /api/tarot/interpret/{reading_id}` · `GET /api/tarot/history` · `POST /api/tarot/outcome/{reading_id}`

Карта и разборы — `app/api/routers/chart.py` `GET/POST /api/chart` · `GET /api/matrix` · `POST /api/compat` (балл) · `POST /api/compat/full` (разбор LLM) · `GET/POST/DELETE /api/partners/{id}` · `GET /api/reports` · `GET/POST /api/reports/{kind}`

Сегодня и небо — `app/api/routers/today.py` `GET /api/today` · `GET /api/moon/week?days=7` · `GET /api/sky` · `GET /api/horoscope` · `GET /api/horoscope/all`

Профиль — `app/api/routers/profile.py` `GET /api/me` · `POST /api/profile` · `GET /api/personas` · `GET /api/referral` · `GET/DELETE /api/memories[/{id}]` · `GET /api/faq` · `GET /api/health`

Резерв (бэкенд готов, экранов пока нет; подключаются фичами при необходимости) `app/api/routers/diary.py`: `GET/POST /api/diary`, `GET /api/diary/prompt` `app/api/routers/practices.py`: `GET /api/practices`, `GET /api/practices/{code}`, `POST /api/practices/{code}/start|/done|/stop` `app/api/routers/shop.py`: `GET /api/shop`, `POST /api/shop/invoice`, `POST /api/shop/crystals`, `POST /api/shop/promo`, `POST /api/shop/web-checkout`, `GET /api/shop/orders`, `GET /api/shop/crystals/history` `app/api/routers/share.py`: `GET /api/share/reading/{id}.png`, `GET /api/share/today.png`, `GET /api/share/enabled`

5. Премияльные механики (детальная проработка)

5.1 3D-переворот карт Таро (реализован)

- **Спека (CSS `.tarot-card`):** `88×136px` (колода ~5:7), `perspective: 1200px` на `.tarot-zone`, `transform-style: preserve-3d`, лицо/рубашка — два `.face` с `backface-visibility: hidden`, рубашка — градиент `#2a214f-#151238` с золотой каймой и мотивом «ORACLE», лицо `rotateY(180deg)`.
- **Тайминг:** переворот **700ms**, `cubic-bezier(0.22,1,0.36,1)` (`--ease`); класс `.flipped` → `rotateY(180deg)`.
- **Порядок:** клиентка переворачивает сама, тапом (Haptic light). Пока не все перевёрнуты — кнопки трактовки нет.
- **Доступность:** `prefers-reduced-motion` → карта появляется мгновенно (`crossfade ≤150ms`), без вращения.
- **RTL:** вращение вокруг оси Y, центр — горизонтальная середина; карта не едет.

5.2 Натальное колесо (реализовано упрощённо)

- Сейчас — стилизованное **мини-колесо** (`chart-wheel`): круг с `radial-gradient`, пунктирное внутреннее кольцо, в центре символ Солнца (`sun.symbol`) + знак + АС. Полный SVG-колесо с домами и аспектами — **будущее** (данные есть: `chart.planets/houses/aspects`).
- Планеты — списком `planet-line`: глиф знака (`SIGNS`), имя, `знак · дом N`, «`♁`» если ретроградная.
- **Источник:** `GET /api/chart` → `planets, houses, aspects, sun, ascendant, mc`.

5.3 Матрица Судьбы (реализовано списком)

`GET /api/matrix` → виджет списком: ключи `personal/destiny/love/money`, каждая строка `label · arcana` + число `n`. Полная октаграмма на 7 узлов — **будущее** (бэкенд уже отдаёт узлы). Итоговый разбор — вопросом агенту.

5.4 Совместимость (реализована как LLM-разбор)

Форма (имя + дата) → `POST /api/compat/full` → ответ Астролога в ленте. Балл (`POST /api/compat` → `score, breakdown`) — **резерв** для «спидометра» на будущее; формула живёт на сервере (`skills._compat`), чтобы чат и приложение не расходились.

5.5 Карточки для сторис (резерв)

`GET /api/share/reading/{id}.png` и `GET /api/share/today.png` возвращают **PNG**, отрендеренный на бэкенде (флаг `share_cards` → `GET /api/share/enabled`). Фронт: кнопка «В сторис» → `Telegram.WebApp.shareMessage` , fallback — `navigator.share` /скачать. Если `share_cards=false` — кнопку не показываем.

5.6 Стрик (резерв)

`diary_streak` из `/api/me` уже показан в статистике профиля. Кольцо-прогресс, «полоса дней» и практики (`/api/practices`) — **будущее** (бэкенд готов).

5.7 Карта дня — ритуал открытия

На «Сегодня»: hero-орба со свечением + карта дня выходят из тени (scaling + crossfade, ~500ms), затем прогноз «договаривает» (fade). Повторное открытие в тот же день — сокращённый кроссфейд (~150ms), не раздражает. `prefers-reduced-motion` — без автоанимаций.

6. Сквозные паттерны

6.1 Онбординг (ведёт БОТ, Mini App встречает готовых)

Факт контракта (важно): ввод даты рождения, времени, города, «как меня звать» и флаг `onboarded` ставит **бот** через FSM `app/bot/onboarding.py` . В API **нет** эндпоинта захвата этих полей — `POST /api/profile` меняет только `oracle_name/persona/morning_push/tz/goal` . Исключение — время и город рождения для **натальной карты** можно добавить в чате через `POST /api/chart` (§4.5), но `birth_date` — только из бота.

- **Триггер:** `me.onboarded=0` .
- **Что показывает миниап:**
 - Карточка-приветствие: «Оракул готов тебя послушать. Чтобы я построила твою карту, нам надо познакомиться в чате», кнопка **«Продолжить в боте»** → деп-линк `https://t.me/<bot_username>?start=` (бот сам продолжит FSM).
 - Ниже — мягкая апертюра того, что ждёт в боте (короткий абзац: дата рождения + город → «живая карта»).
 - Повторный визит до завершения — та же карточка (стор помнит).
 - **После завершения:** при следующем открытии `onboarded=1` → обычный вход.
 - **Состояния:** `onboarded=0` — единственное состояние паттерна; ошибка получения `/me` — обычный retry; повторного ввода нет.

6.2 Пейволл и оплата Stars (без «магазина»)

- **Принцип:** никогда «дверь в лицо». Лимит/подписка — **карточка продолжения** в диалоге, а не экран-занавес.
- **Пейволл (402/429):** текст `detail` уже человечный и приходит от бэкенда (`DENY_TEXT`: «Доступ завершён Продли подписку...», «Вопросы исчерпаны. Вернись на рассвете или открой поле силой Кристаллов ♦»). Два пути: «Продлить безлимит» (подписка, `POST /api/shop/invoice {plan} → openInvoice`) / «Экстренный вопрос за Кристаллы ♦ (остаток: N)» (цена — `allowance.emergency_cost`).
- **Stars:** `openInvoice(link)` — системная шторка Telegram; callback `invoiceClosed → refresh /me`. **Отмена** — тихо, без «нам жаль».
- **Фиче-флаг `web_payments`** — если вкл, web-оплата второстепенной кнопкой; выкл — не показываем.

6.3 Пуш-уведомления и напоминания

- **Реальность:** доставки пушей фронту нет; есть флаг `morning_push` + время. Делает бот. Фронт — только UI выбора и тон.
- **Никогда** не спрашиваем повторно, если уже отказалась (`morning_push=false` в `/me`).
- Форма времени — сетка часов, а не скролл-колёсико.

6.4 Смена образа речи агента

- **3 персоны** (`GET /api/personas , me.persona`): **Мудрая подруга** (friend) · **Таинственная ведунья** (witch) · **Духовный наставник** 𐄂 (mentor). Влияют на тон главного агента `oracle (uses_persona=True)`.
- Смена — `POST /api/profile {persona}`; стор обновляет имя/accent главного агента.
- Визуал под персону: только accent-цвет шапки/имени и кайма пузырей (текст/тон — бэкенд).

6.5 Прод-совместимость: CSP и XSS (реализовано)

- **0 inline-хендлеров** (`onclick / oninput / onkeydown` отсутствуют): все действия — единое делегирование `data-act` на документе (см. низ `app.js`). Под строгим прод-CSP (`script-src 'self' https://telegram.org`, без `unsafe-inline`) интерфейс работает целиком.
- **Серверный текст** (история, ответы LLM, отчёты) — через `rich()`: сначала полное экранирование, восстановление только закрытых `<b>/<i>`; `<script>`, `onerror=`, атрибуты остаются текстом. `m.role` в классе пузыря фиксирован (`user|assistant`).
- **Шрифты самохостятся** (`miniapp/fonts/ , oracle-fonts.css`) — внешние стили Google Fonts заблокированы `style-src 'self'`.
- **Cache-busting:** ассеты в `index.html` с `?v=1` (имена без хеша, TTL 1 час на `/static/*` — при деплое версию поднять).

7. Состояния и доступность (общие правила)

- **5 состояний** каждого виджета — loading / empty / error / data / edge. Скелетон (`skeleton` — shimmer-пульс) и loader-ring, не спиннер-вращалка.
- **Error:** текст из `detail` + retry (бэкофф 2/4/8с) + после 3-й попытки — «Связаться с поддержкой» + ID.
- **Edge:** длинные строки → `overflow-wrap` + усечение «...»; проверка кириллицы/RTL.
- **Contrast:** текст $\geq 4.5:1$, крупный текст/иконки $\geq 3:1$; золото только крупно (§DESIGN_SPEC 2.1).
- **Focus/клавиатура:** `:focus-visible` золотое кольцо; Таро доступно с клавиатуры.
- **reduced-motion:** выкл трансляции и 3D, оставить crossfade $\leq 150\text{ms}$; выкл Haptic.
- **Разное:** пустые состояния — фирменная строка + 1 действие, не «каркас-пустышка».

8. Реализовано сейчас / очередь

Реализовано (текущий `miniapp/`)

- [x] Каркас: sticky-шапка + нижний таб (Сегодня/Агенты/Профиль), звёздное небо, glass-карточки.
- [x] Экран «Сегодня»: hero + прогноз + карта дня + кольцо агентов.
- [x] Хаб агентов с фиши-чипсами (реестр `FEATURES`).
- [x] Чат: лента, отправка, «печатает...», очистка треда, подсказки.
- [x] Фиша Таро: вопрос → схема → раздача → 3D-переворот → трактовка → история (модал).
- [x] Фиша Натальной карты: постройка в чате (`POST /api/chart`) → виджет → Профиль.
- [x] Фиши-виджеты: Лунная неделя, Матрица, Прогноз, Совместимость.
- [x] Профиль: статистика, данные, карта, расклады, разборы, память.
- [x] HTTP-обвязка: `X-Init-Data` , `dev_user` , ошибки из `detail` (§2.1–2.2).

Очередь (бэкенд готов — фронт подключает фишами)

- [] Пейволл-карточка 402/429 внутри чата (Кристаллы ✦ / подписка) — сейчас текст из `detail` .
- [] Лавка: подписка/товары (`/api/shop*`), пополнение Кристаллов.
- [] Смена персоны oracle из чата/профиля (`POST /api/profile {persona}`).
- [] «Сбылось» на раскладах (`POST /api/tarot/outcome/{id}`).
- [] Карточки для сторис (`/api/share/*` , флаг `share_cards`).
- [] Практики (`/api/practices*`) фишей у Наи; дневник (`/api/diary*`) фишей у Мнемо.
- [] Полное натальное колесо (SVG, `houses/aspects`); октаграмма Матрицы; «спидометр» совместимости.

- [] Стрик-визуализация (кольцо-прогресс, «полоса дней»).
- [] Полировка: reduced-motion, фокус, контраст-аудит, edge-состояния.

9. Риски и компенсации

Риск	Компенсация
LLM-долгие ответы (/interpret , /chat/{agent} , /compat/full) таймаутят	«печатает...» + retry-бэкофф + кеш дня/карты в сторе
Нет пушей в API; онбординг ведёт бот	UI настройки morning_push + карточка «заверши в боте» (§6.1)
monthly отчёт не подтверждён	показываем только реальные kind из /api/reports
Вопрос к картам забыт → расклад без контекста	вопрос обязателен перед раздачей (doDraw валидирует)
Золото «кричит» на мелком	лимитируем по DESIGN_SPEC (золото только крупно, текст — светлый)
birth_time неверного формата	валидирует бэкенд (ЧЧ:ММ), фронт показывает detail

10. Глоссарий

- **Фича-чипс / feature-chip** — кнопка-функция агента, живёт в диалоге (FEATURES).
- **Чат-виджет (chat.pending)** — системное сообщение-форма в ленте (расклад, карта, форма совместимости).
- **Пламя/лимит** — родственно allowance (не путать с фламентом: в коде нет «flames»).
- **Кристаллы ✦** — crystals (валюта экстренного/быстрой покупки, emergency_cost).
- **allowance** — структура лимита вопросов: plan, limit, used, left, period, extra_questions, crystals, emergency_cost, can_ask .
- **«цифра → раскрытие»** — аккордеон-паттерн против простыни (см. DESIGN_SPEC).

Бизнес и монетизация

Бизнес и монетизация

Модель

Два взаимодополняющих пути: **подписки** (основной продукт) и **отдельные товары** (расклады, отчёты, вопросы, Кристаллы). Плюс **PDF-разборы на маркетплейсах** как канал продаж и трафика. Оплата — Telegram Stars внутри бота и web-оплата (Paddle) в обход комиссии сторов.

Тарифы

Код	Тариф	Цена	Период	Вопросов/день	Память	♦ в подарок
free	♦ Искра	бесплатно	—	1/нед	5	0
trial	♦♦♦ Триал VIP	бесплатно	30 д	3	40	30
guide	♦♦ Путеводная	550 ★ / \$9.99	30 д	1	20	20
vip	♦♦♦ VIP-Оракул	1300 ★ / \$24.99	30 д	3	40	50
vip_year	♦♦♦ VIP на год	8900 ★ / \$179	365 д	3	40	300
concierge	Консьерж	5200 ★ / \$99	30 д	30	80	200

Позиционирование: «guide» — активационный уровень с персональным прогнозом, «vip» — выбор большинства («личный астролог 24/7 дешевле одной консультации»), «concierge» — эксперимент high-ticket с безлимитом, приоритетом и аудио-прогнозами. Цены правятся в админке (каталог), значения по умолчанию — в `app/data/seed.py`.

Товары без подписки

- **Одиночные расклады Таро** от «Одна карта» (75 ★) до «Колесо года» (550 ★).
- **Глубокие отчёты:** разбор натальной карты, Матрицы Судьбы, синастрии, карьеры, годовой прогноз.
- **Вопросы сверх лимита:** +1 (60 ★) и +5 (250 ★).
- **Кристаллы ♦:** экстренная валюта и оплата за сверхлимитные вопросы. Пакеты со скидкой (100/250/600 ♦, лестница монотонна).

Кристаллы

Внутренняя валюта для «экстренной магии»: заплатить за вопрос сверх лимита (20 ♦), получить за реферальные действия и как часть тарифа. Лестница пакетов построена так, что больше — всегда выгоднее (100/250/600 ♦).

Платежи

- **Telegram Stars** — основной путь внутри бота (инлайн-инвойсы). Telegram забирает комиссию ~30–45%.
- **Paddle (web)** — платёжная страница вне Telegram, комиссия 3–5%. Это **рычаг маржи**: включение флага `web_payments` после проверки вебхука на боевом домене. Флаг выключен по умолчанию — сломанный платёжный путь хуже, чем отсутствие пути.
- **Идемпотентность**: вебхуки проверяются по подписи и журналу `webhook_events` — повторная доставка события никогда не выдаёт товар дважды. Подписки не автопродлеваются; доступ живёт по дате `sub_until`.

Юнит-экономика

Цель — измеримая себестоимость и здоровая маржа. Реальную LLM-себестоимость сервис пишет сам на каждый вызов (`cost_usd`), сводка по `purpose` — в панели.

- **LLM-себестоимость**: цель < 30% цены тарифа. При \$24.99 это < \$7.5/мес на платящую; при трёх вопросах в день бюджет вопроса ≈ \$0.08 — достигается за счёт экономии на lite-вызовах, батчинга и кеша прогноза.
- **CAC**: по бесплатным каналам ≈ 0 (каналы-спутники, рефералка, органика); PDF-маркетплейсы ≈ себестоимость разбора; платная реклама — после первых тестов.
- **ARPU** = выручка / активные платящие.
- **LTV** = ARPU × средняя жизнь (стартовый ориентир 4–6 мес) × маржа (выручка – LLM – комиссия).
- **Порог здоровой экономики**: $CAC < LTV/3$ и $LLM < 30\%$ цены тарифа. Оба — красные флаги перед масштабированием платного трафика.

Метрики и воронка

- **Воронка**: `/start` → завершил онбординг → первый вопрос → второй день → подписка → D7-ретеншн → продление.
- **Метрики**: DAU/WAU, активация (onboarded), retention D1/D7/D30, конверсия в оплату, churn, возвраты, когорты по `source` (канал привлечения).
- **Каналы продаж, которые видны в аналитике**: `source` на пользователя, `promo.batch` на промокод — видно, какой листинг/канал приводит платящих.
- Если активация или D1-ретеншн низкие — это в приоритете дизайн и онбординг, а не инженерия.

Проверки перед запуском

- [] Все вызовы LLM пишут `cost_usd` — сводка «Себестоимость» в панели.
- [] Цены витрины совпадают с таблицей тарифов (панель → Каталог).
- [] Paddle: тест вебхука на боевом домене → флаг `web_payments` ON.
- [] После 30 дней продаж — пересчёт ARPU/LTV на реальной выборке и подтверждение $CAC < LTV/3$.

Операции и эксплуатация

Операции и эксплуатация

Как проект живёт на сервере: деплой, мониторинг, бэкапы, запуск и масштабирование. Для DevOps/админов.

Состав и принципы

Процессов ровно два — **бот** и **API**, это принципиальное решение:

- Планировщик, утренние прогнозы, рассылки и автопостинг гороскопов живут **только** в процессе бота. Второй экземпляр бота отправил бы каждое сообщение дважды.
- API масштабируется свободно, но держится один воркер — лимитеры (запросов и исходящего Telegram) живут в памяти процесса. Синхронизация появится с переходом на PostgreSQL + Redis при росте до 30–50k пользователей.

SQLite (WAL) на одном VPS уверенно держит 10k пользователей; схема написана переносимо — при росте меняется только слой сессии.

Деплой

```
docker compose -f infra/docker-compose.yml up -d --build
```

В комплекте:

- **бот + API** (Dockerfile);
- **Caddy** — автоматический https, домен впиши в `infra/Caddyfile` и в `.env` → `WEBAPP_URL` ;
- **healthcheck** бота и API — зависший контейнер перезапускается сам;
- **ежедневный бэкап** базы (`sqlite3 .backup` , хранение 14 дней).

Перед боевым запуском: `DEV_MODE=0` , свой `BOT_TOKEN` , заполненный `ADMIN_ID` , `WEBAPP_URL=https://домен` .

Мониторинг

- **Sentry** (`SENTRY_DSN`): ловит все необработанные исключения бота и API — включается первым, без него «багов нет» просто не видно.
- **UptimeRobot** мониторит внешний `https://домен/api/health` .
- **Алерты**: падение процесса (контейнер перезапущен), рост 5xx > 1% за 5 минут, backlog рассылок, доля фолбэков/офлайна LLM и средняя латентность, диск > 80%, размер бэкапов.
- **Логи**: `docker logs` с ротацией (json-file). На старте этого достаточно.

Бэкапы

- Локальный `.backup` ежедневно (compose, 14 дней).
- **Off-site**: выгрузка на S3-совместимое хранилище или второй VPS — локального бэкапа мало для персональных данных.
- **Проверка восстановления** раз в месяц: восстановить бэкап в пустую БД, прогнать `PRAGMA integrity_check` + `selfcheck`. Непроверенный бэкап — это отсутствие бэкапа.

CI/CD

GitHub Actions: lint (ruff), pytest + `selfcheck` на каждый PR, сборка Docker, метки на теги → деплой на VPS по SSH.

- **Staging**: отдельный инстанс (та же compose) с тестовым ботом-дубликатом и `DEV_MODE=1`. На нём — ручные проверки и нагрузочные прогоны.
- **Релизная дисциплина**: заморозка деплоев за 48 ч до запуска; деплой утром в рабочий день; откат = `docker compose` на предыдущий тегированный образ.

Запуск (runbook)

1. **За 48 ч**: заморозка деплоев. Проверить конфиг на боевом: `DEV_MODE=0`, `BOT_TOKEN`, `ADMIN_ID`, `WEBAPP_URL`, LLM-ключи, `PADDLE_WEBHOOK_SECRET` (если включаем). `selfcheck` на сервере — все .
2. **За 24 ч**: ручной тест-план на staging (онбординг, оплата Stars и Paddle, рассылка на себя), тестовая оплата, бэкап проверен восстановлением.
3. **За 2 ч**: включить мониторинг (Sentry, алерты), проверить логи, снять снапшот размеров БД.
4. **День X**: маркетинг по плану (`docs/MARKETING.md`). Наблюдать: Sentry (рост ошибок — фикс/откат по протоколу), планировщик (уложился ли утренний тик), LLM (фолбэки, себестоимость, 429), БД (размер, `database is locked`).
5. **При проблеме**: уведомить в админ-чат, применить откат на предыдущий образ, НЕ держать систему в полуробочем состоянии. Проблема → задача в бэклог → фикс после стабилизации.

Целевые SLO

- Ответ бота $p95 < 10$ с, $p99 < 30$ с (включая LLM-цепочку и фолбэк).
- API $p95 < 400$ мс, $p99 < 1,5$ с (кроме генераций); 5xx < 0,1%; аптайм 99,5%.
- Утренний прогноз до 09:30 по локальному времени; рассылки — каждый таргет ровно один раз, доставка за час.
- Пик 10 000 юзеров/сутки, до 4 000 DAU, до 2 000 одновременных в чате — без деградации.

Масштабирование

- **30–50k пользователей** → PostgreSQL с репликой + Redis-лимитеры (схема переносима, меняется слой сессии и тип id).
- Перед масштабированием обязательны: off-site бэкап, пересчитанная юнит-экономика ($CAC < LTV/3$, $LLM < 30\%$ тарифа), нагрузочные прогоны.

Проверки перед боевым запуском

- [] `DEV_MODE=0`, боевые токены, домен, https (Caddy).
- [] Sentry подключён, алерты настроены, off-site бэкап проверен восстановлением.
- [] Денежные пути идемпотентны (тест «повторный платёж не выдаёт товар дважды»).
- [] Рассылки без дублей; аудитория не голодает (пагинация).
- [] Ручной тест-план пройден на staging; runbook прочитан и понятен.

Право и приватность

Право и приватность

Сервис собирает дату, время и город рождения — это персональные данные. Ниже — юридический контур продукта: согласие, политика, оферта, право на удаление. Тексты короткие и человечные, как и продукт; перед публикацией их должен прочитать юрист — это не замена юридического заключения.

Что собираем и зачем

- **Имя, username Telegram** — чтобы обращаться и идентифицировать аккаунт.
- **Дата, время и город рождения** — для расчёта натальной карты и прогнозов. Без них продукт не работает.
- **Переписка и «память»** — чтобы Оракул помнил пользовательницу и отвечал персонализированно.
- **Оплаты** — без карт: платёжные данные у провайдеров (Telegram Stars / Paddle), сервису приходит только факт и сумма.
- **Технические логи** — безопасность, диагностика, аналитика (обезличенная).

Хранение — на сервере оператора; передача третьим лицам — только платёжным провайдерам и облаку; продажа данных исключена. Срок хранения — пока активна подписка + разумный срок для бухгалтерского учёта.

Согласие (в онбординге)

Продолжая, ты принимаешь правила сервиса и соглашаешься на обработку персональных данных (дата рождения, имя, переписка) для предоставления астрологических прогнозов, памяти и рассылок, а также на получение сообщений от бота. Отменить можно в любой момент командой `/delete_me`.

Текст живёт в приветствии бота и правится в админке.

Право на удаление: `/delete_me`

Команда `/delete_me` (подтверждение словом «УДАЛИТЬ») анонимизирует аккаунт: имя, дату рождения, карту, дневник, память и переписку очищаются; аккаунт помечается удалённым. Необратимо нужно для финансового учёта (записи об оплатах) остаётся — это обязательство по закону о бухгалтерских книгах, про это явно написано в политике.

Политика конфиденциальности (коротко)

1. **Что собираем:** имя, username, дату/время/город рождения, переписку, оплаты (без карт), технические логи.
2. **Зачем:** личные прогнозы, память, оплата тарифа, рассылки по выбору, безопасность и аналитика (обезличенная).
3. **Где живёт:** серверы оператора; передача третьим лицам — только платёжные провайдеры и облако; без продажи данных.
4. **Как долго:** пока активна подписка + разумный срок для учёта; `/delete_me` сокращает до обязательного минимума.
5. **Права:** доступ, копия, исправление, удаление — через поддержку или `/delete_me`.
6. **Контакты:** email/Telegram поддержки оператора — вписать перед запуском.

Оферта (краткое изложение)

- **Предмет:** доступ к чат-боту «Оракул», подписки, расклады и разборы.
- **Оплата:** Telegram Stars или web (Paddle). Подписка не автопродлевается — доступ по дате `sub_until`.
- **Возвраты:** возврат, если продукт не оказан (нет доступа или ошибка выдачи), — в течение 14 дней по обращению в поддержку. Качество прогноза — творческий продукт, «не понравилось» возвратом не считается (как консультация).
- **Ограничение ответственности:** прогнозы — для самопознания и вдохновения, не заменяют врача, психолога, юриста или финансового консультанта. Кризисные сообщения — в раздел поддержки.
- **Правонарушения:** остигматизация, спам, насилие → бан без возврата.
- **Изменения оферты:** публикуются здесь; продолжение использования = согласие.

Дисклеймер

«Оракул создан для самопознания и вдохновения. Он не заменяет врача, психолога и юриста» — встроен в настройки сервиса и показывается в онбординге и FAQ. Астрологические заявления не выглядят как гарантии (особенно про деньги и здоровье). Возраст 18+ — зафиксирован в оферте.

Кризисный протокол

Сообщения о суициде, самоповреждении и насилии не доходят до модели — пользовательница получает поддержку и контакты помощи, лимит не тратится. Контакты локальны по странам (РФ, Казахстан, Беларусь, Украина, международный список) и правятся в панели.

Перед запуском (чек-лист юриста)

- [] Вписать реквизиты оператора и контакты поддержки.

- [] Проверить юрисдикцию (РФ 152-ФЗ / GDPR, если клиентки в ЕС).
- [] Подтвердить политику возвратов и окно 14 дней.
- [] Убедиться, что `/delete_me` действительно проходит весь путь анонимизации.
- [] Разместить политику и оферту на лендинге и в /help.